Laboratory report of Nanjing University of Information Science and Technology

Experiment Title: Containerization with Docker

Data：2026/5/24

Class：1

Name：Xue Zhengyang

ID：202383910028

一、Experimental purpose

Learn how to package a web application into a Docker Image.

Understand the concept of Immutable Infrastructure.

Practice mapping network ports between a Cloud Container and the Host.

二、Experiment Contents

We will now build a Docker image for your previous index.html

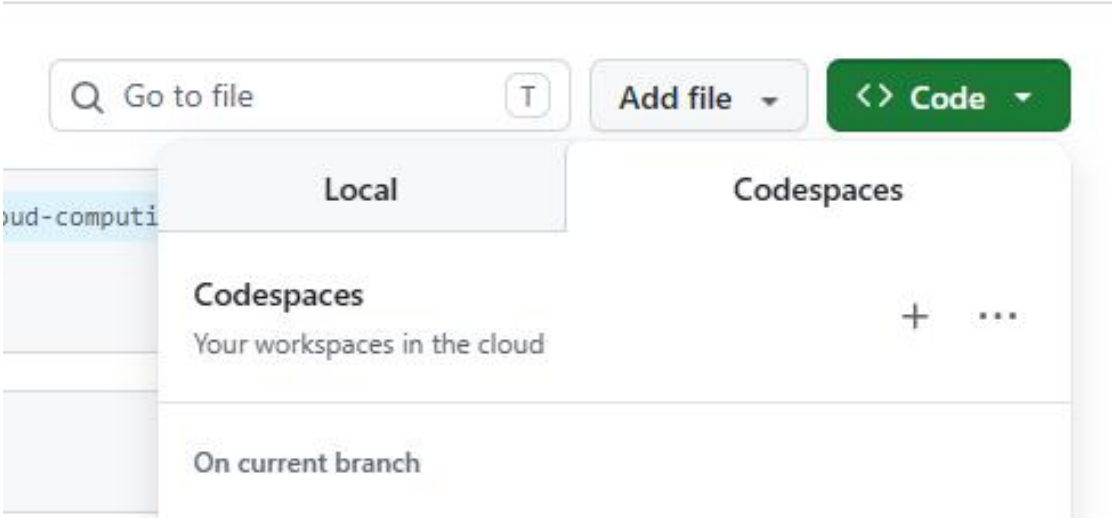
三、Detailed steps of the experiment

Lab Report Requirements: Clearly present the procedure step by step, with corresponding screenshots provided for each stage. Screenshots must demonstrate that they were taken on your own computer, for example by including visible system time, desktop environment, or other identifiable personal elements.

✓ Environment Setup

1. Open your GitHub repository.

2. Click the Code button and select Codespaces.



3. Click Create codespace.

Step 1: Verify Docker Environment

In your Codespaces terminal (at the bottom), type the following command to ensure the Docker daemon is running:

Bash： docker version

Observation: You should see both Client and Server version information.

Step 2: Create a Dockerfile

A Dockerfile is a script containing instructions on how to build your image.

In the file explorer on the left, click New File.

Name it exactly Dockerfile (case - sensitive, no file extension).

Paste the following code:

Use Nginx (a popular web server) as the base image

FROM nginx:alpine

Copy your local index.html into the Nginx web directory inside the container

COPY index.html /usr/share/nginx/html/index.html

Tell Docker that the container listens on port 80 at runtime

EXPOSE 80

Step 3: Build the Docker Image

Run the following command to "bake" your code into a reusable image:

Bash： docker build -t cloud-lab-image:v1 .

Cloud Concept: The . at the end tells Docker to look for the Dockerfile in the current folder.

Step 4: Run the Container

Now, launch a "Running Instance" (Container) based on your image:

Bash: docker run -d -p 8080:80 --name my-web-container cloud-lab-image:v1

Explanation: -p 8080:80 maps your Cloud VM's port 8080 to the Container's port 80.

Step 5: Access the Web Server

Codespaces will detect a new port is active. A pop-up will

appear in the bottom right: "A service is running on port 8080." Click Open in Browser.

Observation: You should see your personal index.html being served, but this time it is coming from inside a Docker container!

Step 6: Testing Container Immutability

In Cloud Computing, containers are immutable (unchangeable). This experiment proves why we "build once and run anywhere."

- Modify the Code: Go to your index.html file and change the background color or the text (e.g., change "Success!" to "Updated!").
- Refresh the Browser: Go back to the tab where your site is running on port 8080 and refresh.
- Observation: The website did not change.
- The Concept: Even though you changed the source file, the Image running inside the container is a static snapshot of the past. To apply changes, you must rebuild the image and restart the container.

Step 7: Exploring the Container Interior (Shell Access)

A container is like a mini-computer. Let’s "step inside" to see how the cloud organizes files.

Enter the Container: Run the following command to open a

terminal inside your running container:

Bash: docker exec -it my-web-container sh

Navigate Files

Once the prompt changes (usually to a #), type these commands:

ls /usr/share/nginx/html/

cat /usr/share/nginx/html/index.html

Exit

Type exit to return to your main Cloud VM terminal.

The Concept

This demonstrates Isolation. The container has its own file system, separate from your Codespace.

Step 8: Cloud Resource Monitoring

Cloud providers charge based on resource usage. Let's see how "lightweight" containers really are compared to traditional Virtual Machines.

1. Check Stats:

   Run the command: bash: docker stats

2. Analyze:

   Look at the MEM USAGE and CPU % columns in the real-time table.

3. Observation:

   Notice that a full web server is likely using less than

5MB of RAM. A traditional VM would require at least 512MB to 1GB to do the same task.

The Concept:

This is the Efficiency of the cloud. Containers share the host's OS kernel, allowing you to run hundreds of instances on a single server.

(Press Ctrl+C to stop the monitoring).

Step 9: Scaling and Port Management

In the cloud, "Scaling" means running multiple copies of the same service to handle more traffic.

1. Run a Second Instance:

   Launch another container using the same image but a different host port:docker run -d -p 8081:80 --name second-container cloud-lab-image:v1

2. Verify:

   Check the "Ports" tab in Codespaces. You should now have two active ports:8080 and 8081. Open both to see identical sites.

3. Intentional Error:

   Try running a third container using port 8080 again.

- Observation:

  Docker will throw an error:"Bind for 0.0.0.0:8080 failed: port is already allocated."

The Concept:

This is Scalability. We can spin up identical containers instantly, but we must manage the network traffic (ports) correctly.

Step 10: Lab Cleanup

Professional cloud engineers always clean up resources to save costs and keep the environment tidy.

1. Stop and Remove: Stop the processes and delete the container instances:

Bash: docker stop my-web-container second-container
docker rm my-web-container second-container

2. Verify Cleanup: Run the following command to ensure no containers are currently running:bash: docker ps

The Concept: In a "Pay-as-you-go" cloud model, stopping unused resources is essential to prevent unnecessary billing.

Step 11: Use git to save your changes.

- git add . (Gather all your lab changes)
- git commit -m "Finished Lab 02 " (Seal the package)
- git push (Ship it to GitHub for permanent storage)

四、Reflection

1. Problems Encountered and Proposed Solutions

2. Explain why your website didn't update automatically after editing the code, and why this is actually a "feature" in cloud computing.

Laboratory Report of Nanjing University of Information Science and Technology

Experiment Title: Containerization with Docker
Data：2026/5/24
Class：1
Name：Xue Zhengyang
ID：202383910028

1. Experimental Purpose

The purpose of this experiment was to learn how to package a web application into a Docker image, understand the concept of immutable infrastructure, and practice mapping network ports between a cloud container and the host machine.

2. Experimental Contents

This experiment involved building a Docker image for a previous index.html file, running the container in a cloud development environment, testing container immutability, exploring the container filesystem, monitoring resource usage, testing scalability through port management, and properly cleaning up resources.

3. Detailed Steps and Observations

Environment Setup

I opened my GitHub repository and created a new Codespace by clicking the Code button and selecting Create codespace on main. After the environment initialized, I proceeded with the following steps.

Step 1: Verify Docker Environment

In the Codespaces terminal, I typed the command docker version. The output displayed both the Client and Server version information, confirming that the Docker daemon was running properly inside the cloud development environment.

Step 2: Create a Dockerfile

I created a new file named exactly Dockerfile with no file extension and saved it in the root directory. The file contained three instructions: FROM nginx:alpine to use a lightweight web server base image, COPY index.html /usr/share/nginx/html/index.html to copy my local web page into the container, and EXPOSE 80 to document that the container listens on port 80.

Step 3: Build the Docker Image

I executed docker build -t cloud-lab-image:v1 . in the terminal. The dot at the end specified the current directory as the build context. Docker successfully downloaded the nginx:alpine layers and built the custom image tagged as cloud-lab-image:v1.

Step 4: Run the Container

I launched a container using docker run -d -p 8080:80 --name my-web-container cloud-lab-image:v1. The -d flag ran it in detached mode, and -p 8080:80 mapped the host port 8080 to the container port 80.

Step 5: Access the Web Server

Codespaces automatically detected the active service on port 8080 and displayed a notification. I clicked Open in Browser and successfully viewed my personal index.html page being served from inside the Docker container.

Step 6: Testing Container Immutability

I opened index.html in the editor and changed the background color and some text

content, then saved the file. When I refreshed the browser tab running on port 8080, the webpage remained unchanged. This demonstrated that the running container is a static snapshot of the image built earlier. The source file modification on the host did not affect the running container because containers are designed to be immutable.

Step 7: Exploring the Container Interior

I accessed the running container shell using docker exec -it my-web-container sh. Inside the container, I ran ls /usr/share/nginx/html/ and cat /usr/share/nginx/html/index.html. The output showed the original version of the HTML file, confirming that the container maintains its own isolated filesystem separate from the host Codespace. I exited by typing exit.

Step 8: Cloud Resource Monitoring

I ran docker stats to observe real-time resource consumption. The my-web-container entry showed extremely low memory usage, approximately 2 to 5 megabytes, and minimal CPU utilization when idle. This demonstrates the efficiency of containers compared to traditional virtual machines, which typically require 512MB to 1GB of memory to perform the same task. I pressed Ctrl+C to stop the monitoring.

Step 9: Scaling and Port Management

To test scalability, I launched a second container using docker run -d -p 8081:80 --name second-container cloud-lab-image:v1. The Ports tab in Codespaces showed both 8080 and 8081 active, and both displayed identical websites. Next, I intentionally attempted to run a third container on port 8080 again. Docker returned the error: Bind for 0.0.0.0:8080 failed: port is already allocated. This proved that while we can scale horizontally by creating multiple container instances, each instance must bind to a unique host port.

Step 10: Lab Cleanup

I stopped and removed the containers using docker stop my-web-container second-container followed by docker rm my-web-container second-container. I

verified the cleanup by running docker ps, which returned an empty list showing no running containers. This step reflects the professional practice of resource management in pay-as-you-go cloud environments.

Step 11: Git Version Control

I saved all changes using git add ., then committed with git commit -m "Finished Lab 02", and finally pushed to GitHub with git push to ensure permanent storage of the lab work.

4. Problems Encountered and Proposed Solutions

During the experiment, I encountered several issues. First, when I initially modified the index.html file and refreshed the browser, I was confused because the changes did not appear. I initially thought there was a caching issue or a problem with the Codespaces port forwarding. After reviewing the lab instructions and re-reading the concept of immutability, I realized that containers do not dynamically reflect host filesystem changes. The solution was to rebuild the Docker image using docker build and start a new container from the updated image.

Second, when testing scalability, I accidentally tried to bind the second container to port 8080 again and received a port allocation error. I solved this by carefully mapping the second container to port 8081 instead, which taught me the importance of unique port management when running multiple instances of the same service.

Third, during the Dockerfile creation, I nearly saved the file as dockerfile.txt because some editors automatically append extensions. I caught this mistake and renamed it to exactly Dockerfile with a capital D and no extension, which Docker requires for automatic detection during the build process.

5. Reflection

Question 1: Why did the website not update automatically after editing the code?

The website did not update because a Docker container is an isolated runtime environment based on a static image snapshot. When I executed docker build, Docker created an image that encapsulated the exact state of my index.html file at that specific moment. The COPY instruction in the Dockerfile copies the file content into the image layer during build time, not during runtime. Therefore, modifying the source file on the host filesystem after the image was built has no effect on the files already stored inside the running container. The container operates with its own independent filesystem layer, completely isolated from the host. To apply changes, the image must be rebuilt so that the new file content is baked into a fresh image, and a new container must be launched from that updated image.

Question 2: Why is this actually a feature in cloud computing?

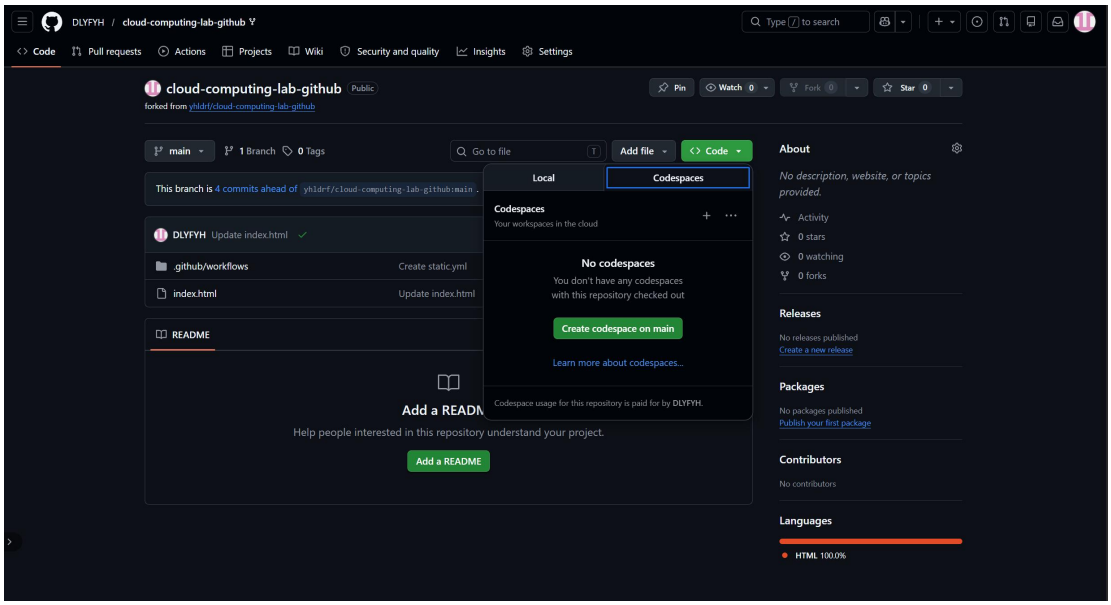
This behavior is a fundamental feature of cloud computing known as immutable infrastructure. Immutability ensures consistency, reliability, and predictability across all deployment environments. Because the running container cannot be accidentally modified by external host changes, developers are guaranteed that the application running in testing is exactly identical to the application running in production. This eliminates the common problem where code works on one machine but fails on another due to subtle environment differences. It also enforces proper version control and deployment practices: every change must go through a documented rebuild and redeploy process, which naturally integrates with continuous integration and continuous deployment pipelines. Furthermore, immutability simplifies rollback procedures. If a new version introduces problems, administrators can instantly revert to the previous stable image rather than debugging unknown state mutations on a live server. Therefore, the inability to auto-update from host file changes is not a limitation but a deliberate architectural choice that enhances stability, security, and operational efficiency in cloud-native systems.

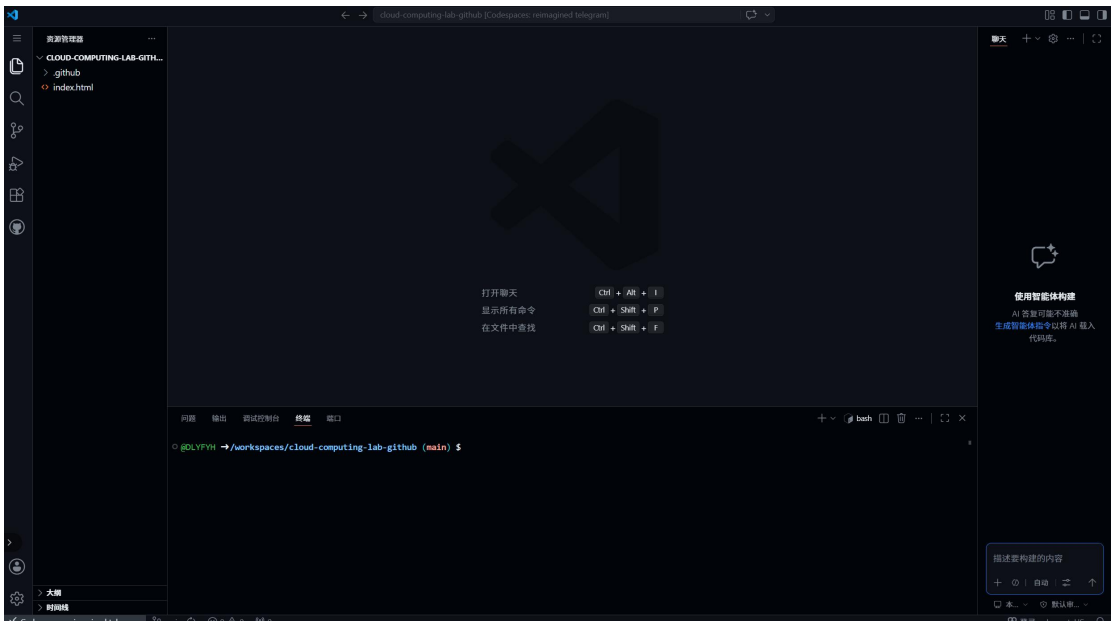
6. Conclusion

This laboratory exercise provided a comprehensive hands-on introduction to Docker containerization and core cloud computing concepts. I successfully built a custom Docker image using a Dockerfile, launched and managed containers, verified network port mapping between host and container, and observed the practical implications of

container immutability. Through direct experimentation, I understood that containers are lightweight, isolated, and efficient runtime environments that share the host operating system kernel rather than requiring full virtual machines. The resource monitoring step clearly demonstrated that containers consume significantly less memory than traditional VMs, which explains why cloud providers can run hundreds of container instances on a single physical server. The scaling and port management exercise revealed both the ease of horizontal scaling and the necessity of careful network port allocation. Finally, the cleanup and Git workflow reinforced professional habits essential for cloud resource management and version control. Overall, this experiment transformed abstract cloud concepts into concrete technical skills, and I now appreciate why immutability and isolation are foundational principles in modern cloud-native application deployment.

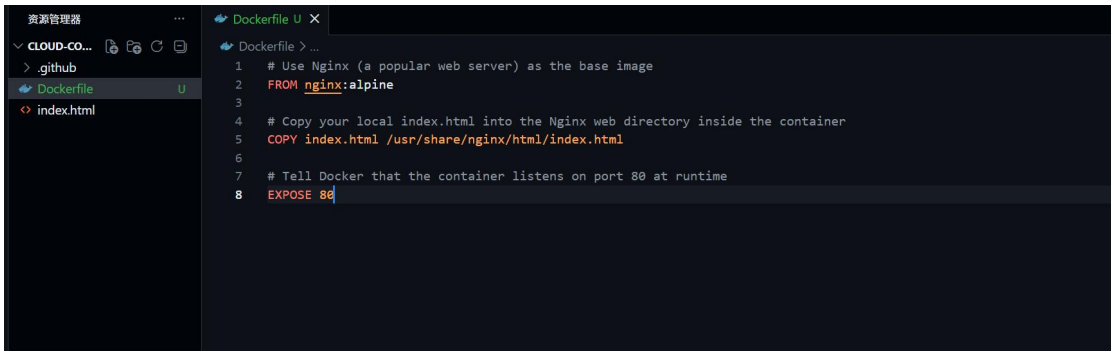
Screenshot:





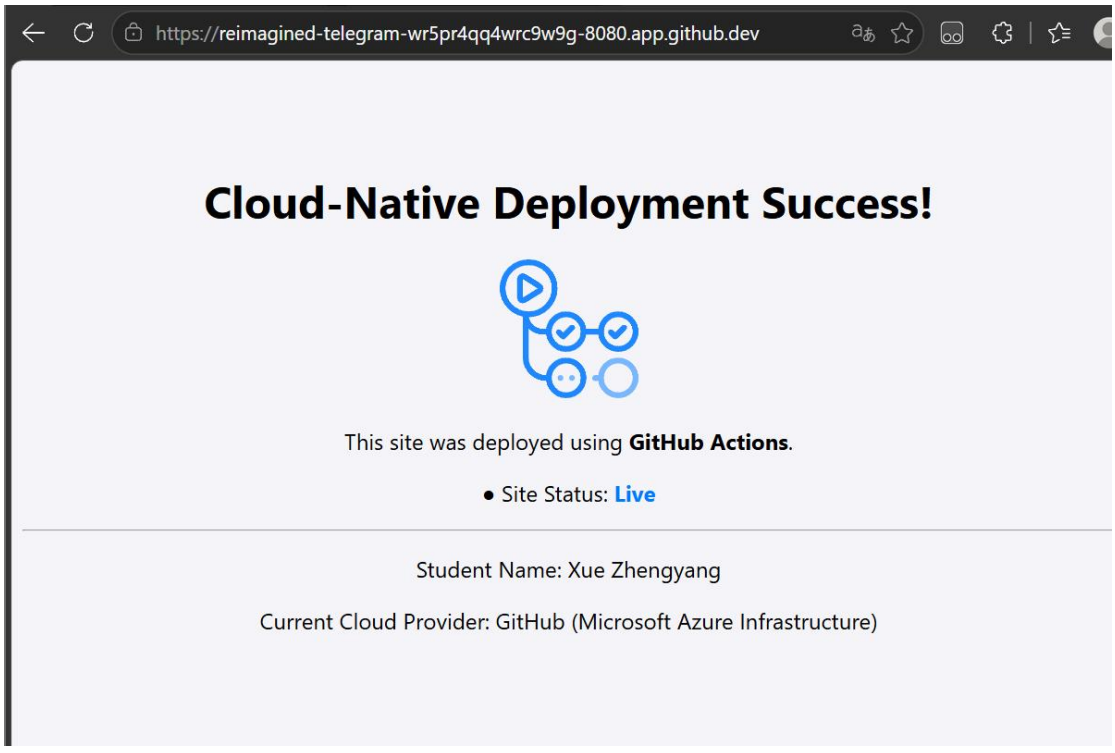
问题 输出 调试控制台 **终端** 端口

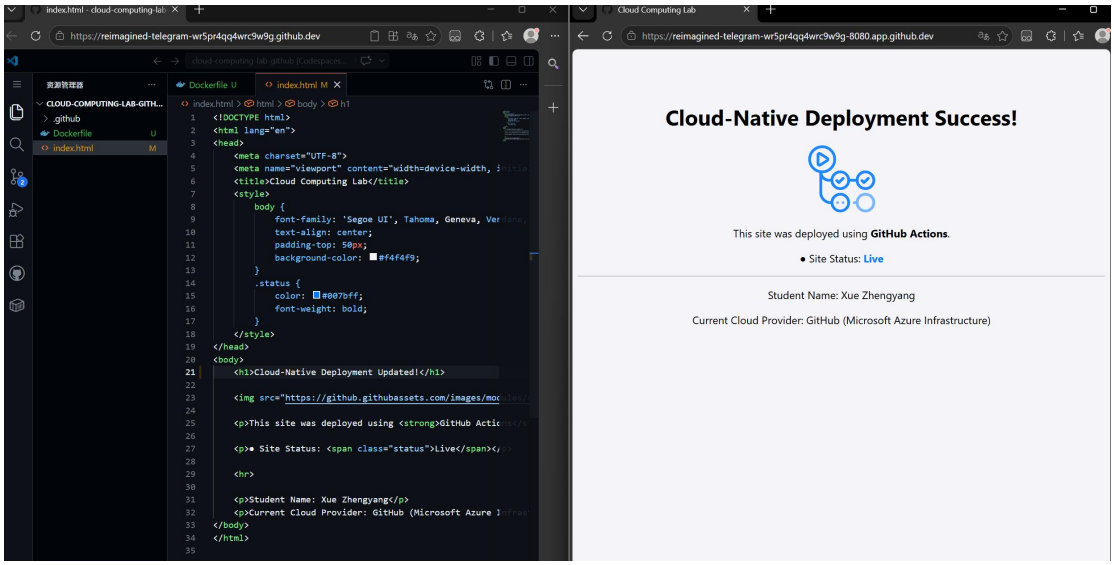
```
@DLYFYH ➜/workspaces/cloud-computing-lab-github (main) $ docker version
Client:
 Version:           29.3.0-1
 API version:       1.54
 Go version:        go1.26.0
 Git commit:        5927d80c76b3ce5cf782be818922966e8a0d87a3
 Built:             Fri Mar  6 18:50:47 2026
 OS/Arch:           linux/amd64
 Context:           default

Server:
 Engine:
  Version:          29.3.0-1
  API version:      1.54 (minimum version 1.40)
  Go version:       go1.26.0
  Git commit:       83bca512aa7ffc1bb4f37ce1107e0d3e3489ad43
  Built:            Fri Mar  6 18:29:13 2026
  OS/Arch:          linux/amd64
  Experimental:     false
 containerd:
  Version:          2.2.1-1
  GitCommit:        dea7da592f5d1d2b7755e3a161be07f43fad8f75
 runc:
  Version:          1.4.0-2
  GitCommit:        8bd78a9977e604c4d5f67a7415d7b8b8c109cdc4
 docker-init:
  Version:          0.19.0
  GitCommit:
@DLYFYH ➜/workspaces/cloud-computing-lab-github (main) $
```



```
@DLYFYH ➜/workspaces/cloud-computing-lab-github (main) $ docker build -t cloud-lab-image:v1 .
[+] Building 6.3s (8/8) FINISHED
 => [internal] load build definition from Dockerfile
 => => transferring dockerfile: 312B
 => [internal] load metadata for docker.io/library/nginx:alpine
 => [auth] library/nginx:pull token for registry-1.docker.io
 => [internal] load .dockerignore
 => => transferring context: 2B
 => [internal] load build context
 => => transferring context: 1.03kB
 => [1/2] FROM docker.io/library/nginx:alpine@sha256:7e8ff0a32da368869608f285124b4375b901401d88f5027865d8f88984d35d38
 => => resolve docker.io/library/nginx:alpine@sha256:7e8ff0a32da368869608f285124b4375b901401d88f5027865d8f88984d35d38
 => => sha256:fbaed3f7fcbe6a0d591ac8601934f1f05252cee42741fde9e950dce55580af18 20.28MB / 20.28MB
 => => sha256:a5008f4a4b257356728e2feaf2b5b9e2054c0c577d324e76abbf14470b5f1cfd 1.40kB / 1.40kB
 => => sha256:e654dbbbb9e15a1fa035d24728aeeb60bc655f4ebd4d6215a496b77a7d104697 1.21kB / 1.21kB
 => => sha256:94d083cf706ab544ec7e9bcaba5c164db87b3d3a56176bf2fca440d535c16b0d 405B / 405B
 => => sha256:de1b677d8c003ce9e558f3a0cd4dec3c035f424ecddd68063043a593ad572257 957B / 957B
 => => sha256:abaae85d1626e429b3f1209aea369c0af9562cc06b5e075c006e0f699bba35f2 1.88MB / 1.88MB
 => => sha256:43f834d60d8af3f133c7e76a202d28cd62cc026a561edca72ee752ef01bfaacd 628B / 628B
 => => sha256:6a0ac1617861a677b045b7ff88545213ec31c0ff08763195a70a4a5adda577bb 3.86MB / 3.86MB
 => => extracting sha256:6a0ac1617861a677b045b7ff88545213ec31c0ff08763195a70a4a5adda577bb
 => => extracting sha256:abaae85d1626e429b3f1209aea369c0af9562cc06b5e075c006e0f699bba35f2
 => => extracting sha256:43f834d60d8af3f133c7e76a202d28cd62cc026a561edca72ee752ef01bfaacd
 => => extracting sha256:de1b677d8c003ce9e558f3a0cd4dec3c035f424ecddd68063043a593ad572257
 => => extracting sha256:94d083cf706ab544ec7e9bcaba5c164db87b3d3a56176bf2fca440d535c16b0d
 => => extracting sha256:e654dbbbb9e15a1fa035d24728aeeb60bc655f4ebd4d6215a496b77a7d104697
 => => extracting sha256:a5008f4a4b257356728e2feaf2b5b9e2054c0c577d324e76abbf14470b5f1cfd
 => => extracting sha256:fbaed3f7fcbe6a0d591ac8601934f1f05252cee42741fde9e950dce55580af18
 => [2/2] COPY index.html /usr/share/nginx/html/index.html
 => exporting to image
 => => exporting layers
 => => exporting manifest sha256:48b6c51e58c4c9c5b3b50c51130fa921c3e21e7c65d50114f3fc826eed3b7bd8
 => => exporting config sha256:b829295e5ff56592b63fde4aebb594771fb9e1a8f88f200d75b7cfae1bac1613
 => => exporting attestation manifest sha256:6f8aeb6176519216af3154dd160ce459ad672e520b8746967fbe31107eba73bc
 => => exporting manifest list sha256:1d018ed6eb3a74559ba1a6f1fe81eb82c784bd88faf7b97c26e1b3d364349206
 => => naming to docker.io/library/cloud-lab-image:v1
 => => unpacking to docker.io/library/cloud-lab-image:v1
@DLYFYH ➜/workspaces/cloud-computing-lab-github (main) $
```

```
@DLYFYH ➜/workspaces/cloud-computing-lab-github (main) $ docker run -d -p 8080:80 --name my-web-container cloud-lab-image:v1
62c5a08c651e2a3f8930a22cc00375974c22b861a064e5976e946fa242d58df5
@DLYFYH ➜/workspaces/cloud-computing-lab-github (main) $
```






```
@DLYFYH ➜/workspaces/cloud-computing-lab-github (main) $ docker exec -it my-web-container sh
/ # ls /usr/share/nginx/html/
50x.html    index.html
/ # cat /usr/share/nginx/html/index.html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Cloud Computing Lab</title>
    <style>
        body {
            font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
            text-align: center;
            padding-top: 50px;
            background-color: #f4f4f9;
        }
        .status {
            color: #007bff;
            font-weight: bold;
        }
    </style>
</head>
<body>
    <h1>Cloud-Native Deployment Success!</h1>

    <img src="https://github.githubassets.com/images/modules/site/features/actions-icon-actions.svg" width="100" alt="Actions Logo">

    <p>This site was deployed using <strong>GitHub Actions</strong>.</p>

    <p>● Site Status: <span class="status">Live</span></p>

    <hr>

    <p>Student Name: Xue Zhengyang</p>
    <p>Current Cloud Provider: GitHub (Microsoft Azure Infrastructure)</p>
</body>
</html>
/ #
```

```
CONTAINER ID   NAME              CPU %     MEM USAGE / LIMIT     MEM %     NET I/O         BLOCK I/O       PIDS
62c5a08c651e   my-web-container   0.00%     3.438MiB / 7.758GiB   0.04%     6.87kB / 4.37kB   20.5kB / 24.6kB   3
```

```
@DLYFYH ➜/workspaces/cloud-computing-lab-github (main) $ docker run -d -p 8081:80 --name second-container cloud-lab-image:v1v1
9f2a48c81cecf87e216e00abc7fe8d5b9498ab63e8d0c9fdaa9ce4c93caf34b7
@DLYFYH ➜/workspaces/cloud-computing-lab-github (main) $
```

```
@DLYFYH ➜/workspaces/cloud-computing-lab-github (main) $ docker run -d -p 8080:80 --name third-container cloud-lab-image:v1
69872c0897c860133a10ee2f3819493f4114150c137b6879542f4bced1915900
docker: Error response from daemon: failed to set up container networking: driver failed programming external connectivity on endpoint third-container (bf25e6ff54a874e5b9c82f0fe96ac0ce0dc
05437726204f62739ac3e9affe0bc): Bind for 0.0.0.0:8080 failed: port is already allocated

Run 'docker run --help' for more information
@DLYFYH ➜/workspaces/cloud-computing-lab-github (main) $
```

```
@DLYFYH ➜/workspaces/cloud-computing-lab-github (main) $ docker stop my-web-container second-container
my-web-container
second-container
@DLYFYH ➜/workspaces/cloud-computing-lab-github (main) $ docker rm my-web-container second-container
my-web-container
second-container
@DLYFYH ➜/workspaces/cloud-computing-lab-github (main) $ docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
@DLYFYH ➜/workspaces/cloud-computing-lab-github (main) $
```

```
@DLYFYH ➜/workspaces/cloud-computing-lab-github (main) $ git add .
@DLYFYH ➜/workspaces/cloud-computing-lab-github (main) $ git commit -m "Finished Lab 02 - Containerization with Docker"
[main 13d9b04] Finished Lab 02 - Containerization with Docker
 1 file changed, 8 insertions(+)
 create mode 100644 Dockerfile
@DLYFYH ➜/workspaces/cloud-computing-lab-github (main) $ git push
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 2 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 550 bytes | 550.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/DLYFYH/cloud-computing-lab-github
   922d23d..13d9b04  main -> main
@DLYFYH ➜/workspaces/cloud-computing-lab-github (main) $
```